FACULTAD DE CIENCIAS EXACTAS Y NATURALES
Ciencia, Tecnologia y Humanismo

Unidad 3

Aprendizaje Supervisado I: Clasificación

Daniela N. Rim

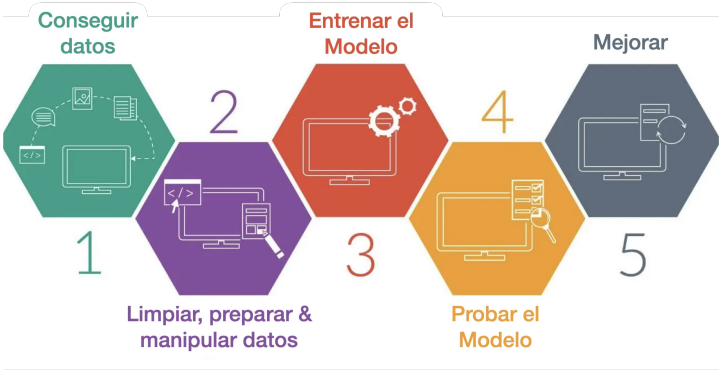
danielarim@handong.ac.kr

16 de Abril, 2026

Repaso

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

Recordemos el flujo general de un sistema de AA:



Flujo típico de trabajo en aprendizaje automático

Inspección inicial del dataset I

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

Con los datos enteros, exploramos:

- **¿Cada fila es una observación y cada columna una variable?**

```
df.head()
df.shape
df.columns
```

- **¿Qué significa cada feature? ¿Cuál es la etiqueta?**

```
data.feature_names
data.target_names
data.DESCR
```

- **¿Qué tipos de datos hay?**

```
df.dtypes
df.info()
```

Inspección inicial del dataset I

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- **¿Hay valores faltantes o duplicados?**

```
df.isnull().sum()
df.duplicated().sum()
```

- **¿Las variables tienen escalas comparables?**

```
df.describe().T
```

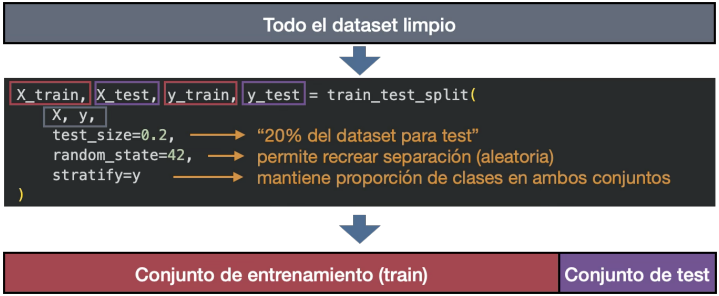
- **¿Hay outliers o distribuciones extrañas?**

```
df.hist()
sns.boxplot(data=df)
```

Conjunto de entrenamiento y test

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

En la práctica 2, hicimos lo siguiente:



- Para pensar: ¿qué te parece que pasaría en el entrenamiento si en vez de usar el 20% (0.2) para test usamos 30%?
- *De ahora en más hasta que hayamos entrenado el modelo, no "miramos" más el conjunto de test.* ¿Por qué no?

Conjunto de entrenamiento y test

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

Cuando entrenamos el modelo, hicimos este proceso:

```
model.compile(
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),
    loss=tf.keras.losses.BinaryCrossentropy(),
    metrics=["accuracy"]
)
```

&

```
history = model.fit(
    X_train_scaled, y_train,
    epochs=100,
    batch_size=32,
    verbose=1
)
```

equivalente a este bucle (for loop):

Inicializar parámetros w, b
for epoch $= 1$ **to** epochs **do**
  for cada (x_i, y_i) en X_train_scaled, y_train **do**
    $y_{\text{hat}} \leftarrow model(x_i)$
    $loss \leftarrow BCE(y_{\text{hat}}, y_i)$
    calcular gradiente ∇L (SGD)
    actualizar w, b (SGD)
  end for
end for

En un epoch...

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

FCEyN

En un epoch (recorrido de todas las muestras en nuestro conjunto), realizamos este bucle:

```
1: for cada (x_i, y_i) en X_train_scaled, y_train do
2:     y_hat ← model(x_i)
3:     loss ← BCE(y_hat, y_i)
4:     calcular gradiente ∇L (SGD)
5:     actualizar w, b (SGD)
6: end for
```

0: Tenemos parámetros $\mathbf{w}, b$
1: Para cada dato y etiqueta del conjunto,
2: Predecimos la etiqueta y_{hat} con $f(x_i)$
3: Medimos el error L
4,5: Optimizamos: ajustamos $\mathbf{w}, b$ para minimizar L y repetimos

¿Cada cuántas muestras optimizamos?

Si tenemos N datos en el conjunto de entrenamiento, ¿cada cuánto actualizamos nuestros parámetros $\theta = (\mathbf{w}, b)$ en la función costo $\mathcal{L}(\theta)$[1]?

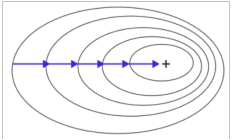
[1] La función costo (a veces J, a veces $\mathcal{L}$) es la suma normalizada de todas las funciones de pérdida $J(\theta) = \frac{1}{N}\sum_{i=1}^{N} L\big(y_i, f(x_i)\big)$

Gradiente por descenso en lote (BGD)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- Usar todo el dataset en cada actualización
- Calcular el gradiente exacto de la función de costo
- En cada epoch:

```
Inicializar θ
while no convergió do
    J(θ) ← (1/N) Σ_{i=1}^{N} L(y_i, f(x_i))
    calcular gradiente ∇_θ J(θ) usando todo el dataset
    θ ← θ − η ∇_θ J(θ)
end while
```



Trayectoria de optimización en BGD, convergencia estable.

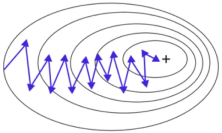
- η: tasa de aprendizaje
- $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$: gradiente usando **todos los datos**

Descenso por Gradiente Estocástico (SGD)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- En lugar de usar todo el dataset, se usa una muestra (x_i, y_i) por actualización
- El gradiente es una aproximación ruidosa del gradiente real
- En cada epoch:

```
Inicializar θ
while no convergió do
    Mezclar (shuffle) el dataset
    for cada (x_i, y_i) en el dataset do
        y_hat ← f(x_i)
        calcular gradiente ∇_θ L(y_i, y_hat)
        θ ← θ − η ∇_θ L(y_i, y_hat)
    end for
end while
```



Trayectoria de optimización en SGD, convergencia ruidosa pero 'rápida'.

- η: tasa de aprendizaje
- $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}; x_i, y_i)$: gradiente usando **una muestra**

Importancia de escala de variables y normalización

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

FCEyN

Decenso por gradiente

ve magnitudes, no qué feature es importante para tomar una decisión

- Ejemplo:
  - $x_A = (2, 1)$, $x_B = (2000, 1)$, Ambas moléculas son R
- Modelo: $f(x) = \sigma(w_1 x_1 + w_2 x_2 + b)$
- Observación:
  - El gradiente respecto a w_j es proporcional a x_j

$$\nabla_w(x_A) \sim (2, 1), \quad \nabla_w(x_B) \sim (2000, 1)$$

- Problema:
  - La variable x_1 domina las actualizaciones
  - El modelo aprende en función de la **escala**, no de la relevancia
  - Gradientes inestables y convergencia más dificultosa
- **Solución:** Normalización o estandarización

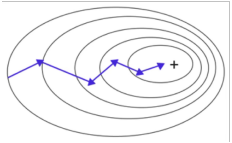
Decenso por gradiente en mini-lotes (MBGD)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- Dividir el dataset en mini-batches (lotes) de tamaño k
- Actualizar parámetros luego de procesar cada batch (lote)

```
Inicializar θ
while no convergió do
    Mezclar (shuffle) el dataset
    for cada mini-batch B de tamaño k do
        ∇ ← 0
        for cada (x_i, y_i) en B do
            y_hat ← f(x_i)
            ∇ ← ∇ + ∇_θ L(y_i, y_hat)
        end for
        ∇ ← (1/|B|) ∇
        θ ← θ − η∇
    end for
end while
```

- B: mini-batch de tamaño k
- Gradiente promedio sobre el batch



Trayectoria de optimización en MBGD, convergencia entre BGD y SGD.

En la práctica....

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- ...usamos MBGD:

```
model.compile(
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),
    loss=tf.keras.losses.BinaryCrossentropy(),
    metrics=["accuracy"]
)
```

&

```
history = model.fit(
    X_train_scaled, y_train,
    epochs=100,
    batch_size=32,
    verbose=1
)
```

- ¿Cómo elegir el $batch_size$ k?

Batch pequeño	Batch grande
Iteraciones rápidas Gradiente más ruidoso Mejor exploración del espacio de parámetros	Gradiente más preciso Iteraciones más costosas Menor ruido (pero no necesariamente mejor convergencia)

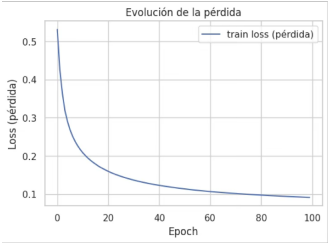
  - En la práctica, batches pequeños suelen funcionar mejor
  - El pasaje de optimización es complejo $\Rightarrow$ gradientes exactos no son críticos
  - Valores típicos: $k = 16, 32, 64$

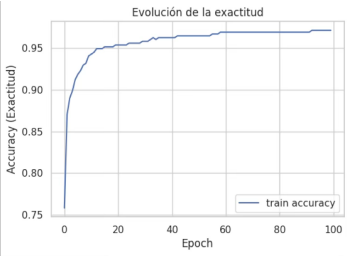
Entrenamiento vs Generalización

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

FCEyN

Entrenamos un modelo (Mini-Batch GD) y monitoreamos:





- La pérdida debería disminuir en promedio (no necesariamente monótonamente)
- La exactitud suele aumentar, pero puede fluctuar

Problema

Alta exactitud en entrenamiento no garantiza buen desempeño en datos nuevos.
Ejemplo: 97% (train) vs 85% (test)

Conjunto de validación

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- **División del dataset:**
  - entrenamiento: usado para actualizar parámetros
  - validación: monitoreo durante entrenamiento (**no** actualiza parámetros)
  - test: evaluación final (generalización)
- Al usar un conjunto de validación:

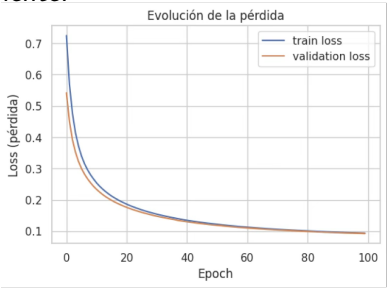
Ventajas	**Desventajas**
Permite monitorear la generalización durante el entrenamiento	Reduce la cantidad de datos disponibles para entrenar
Detecta sobreajuste tempranamente	Puede introducir variabilidad si el dataset es pequeño
Permite ajustar hiperparámetros (ej: learning rate, epochs)	Riesgo de "sobreajustar" al conjunto de validación
Guía decisiones como *early stopping*[2]	La partición puede no ser representativa

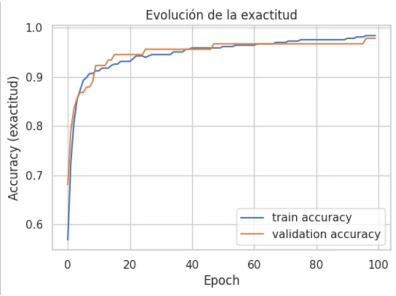
[2] Lo vamos a ver después

Conjunto de validación

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

Idealmente:

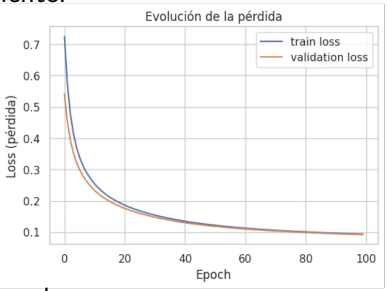


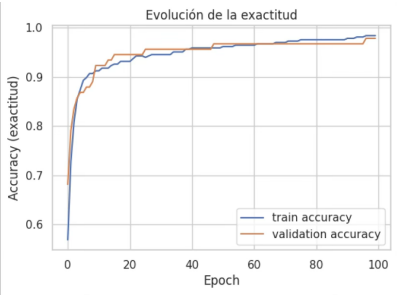


Conjunto de validación

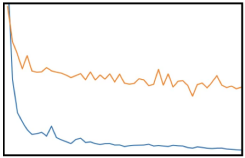
Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

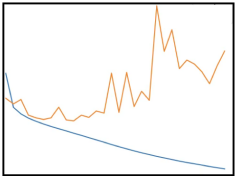
Idealmente:

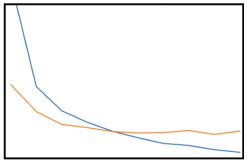




Pero puede pasar:







Parámetros vs Hiperparámetros

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

FCEyN

- **Parámetros (parameters)**
  - Son **aprendidos por el modelo** a partir de los datos
  - Se ajustan durante el entrenamiento (optimización)
  - Definen el modelo final
  - Ejemplos:
    - Regresión lineal: w, b
- **Hiperparámetros (hyperparameters)**
  - Se **definen antes del entrenamiento**
  - Controlan cómo aprende el modelo
  - No se ajustan directamente con gradiente
  - Ejemplos:
    - learning rate (η)
    - número de épocas
    - batch size

¿Cómo elegimos los mejores hiperparámetros?

- Probamos distintas configuraciones $\lambda \in \Lambda$
- Entrenamos en el conjunto de entrenamiento
- Evaluamos en validación $\rightarrow$ elegimos el de mejor desempeño

Cuidado con la terminología

Algunas librerías (por ejemplo, scikit learn) llaman *"parameters"* a argumentos de ciertas funciones. Tener cuidado de no confundir.

Repaso: Categorías de aprendizaje automático

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

	Aprendizaje
Por grado de supervisión humana	• supervisado • no supervisado • semi-supervisado • por refuerzo
Por capacidad de aprendizaje incremental inmediato	• en linea (online) • por baches (batch)
Por enfoque	• Basado en instancias: comparar datos con ejemplos conocidos • Basado en modelos: detectar patrones en los datos de entrenamiento para construir un modelo predictivo

Aprendizaje Supervisado

Introducción

Herramientas de entrenamiento

Optimización

BGD

SGD

MBGD

Conjunto de validación

Parámetros vs Hiperparámetros

Modelos de Clasificación

Regresión logística

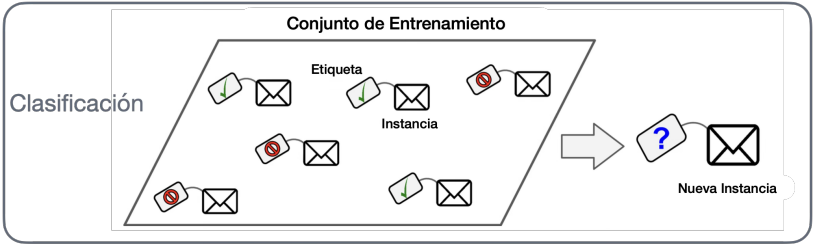
kNN

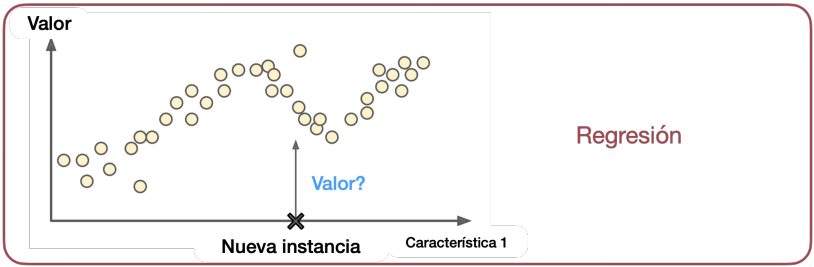
Árboles de decisión

Impulso por gradiente

SVM

Métricas





Formalización de la clasificación supervisada

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

FCEyN

- **Datos:**
  - Dataset: $\{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$
    - $\mathbf{x}_i \in \mathbb{R}^d$ (features)
    - $y_i \in \{1, \ldots, C\}$ (clases)
- **Modelo:**
  - Aprendemos una función: $f : \mathbb{R}^d \to \mathbb{R}^C$
    $f(\mathbf{x}) = (f_1(\mathbf{x}), \ldots, f_C(\mathbf{x}))$
- **Regla de decisión:** $\hat{y} = \arg\max_c f_c(\mathbf{x})$
- **Interpretación:**
  - $f_c(\mathbf{x})$: score para la clase c
  - Puede representar:
    - probabilidades (regresión logística, redes)
    - distancias (kNN)
    - votos (árboles, random forest)
    - márgenes (SVM)

Modelos de clasificación supervisada

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- En esta unidad veremos:
  - regresión logística
  - kNN
  - árboles de decisión
  - random forest (bosques aleatorios)
  - gradient boosting (impulso por gradiente)
  - SVM

Idea clave

Todos los modelos difieren en cómo construyen $f(\mathbf{x})$.

Regresión logística binomial

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- **Modelo:** $P(Y \mid \mathbf{x}) \approx f(\mathbf{x})$
  - $f : \mathbb{R}^d \to [0, 1]$, dos clases (categorías)
  - Entrada: $\mathbf{x} = (x_1, x_2, \ldots, x_d)$
  - Salida (sigmoid):
$$f(\mathbf{x}) = P(Y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$
  - Interpretación:
    - Probabilidad de pertenecer a la clase positiva ($Y = 1$)
    - $P(Y = 0 \mid \mathbf{x}) = 1 - f(\mathbf{x})$
- **Regla de decisión:**
$$\hat{y} = \begin{cases} 1 & \text{si } f(\mathbf{x}) \geq 0.5 \\ 0 & \text{si } f(\mathbf{x}) < 0.5 \end{cases}$$
- **Función de costo (Binary Cross-Entropy):**
$$J(\theta) = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log f(\mathbf{x}_i) + (1 - y_i) \log(1 - f(\mathbf{x}_i))]$$

Regresión logística multinomial

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- **Modelo:** $P(Y \mid \mathbf{x}) \approx f(\mathbf{x})$
  - $f : \mathbb{R}^d \to [0,1]^C$, $\quad C$ clases
  - Entrada: $\mathbf{x} = (x_1, x_2, \ldots, x_d)$
  - Salida (softmax):

$$f_c(\mathbf{x}) = P(Y = c \mid \mathbf{x}) = \frac{e^{\mathbf{w}_c^\top \mathbf{x} + b_c}}{\sum_{j=1}^{C} e^{\mathbf{w}_j^\top \mathbf{x} + b_j}}$$

  - Interpretación:
    - Probabilidad de pertenecer a cada clase
    - $\sum_{k=1}^{C} P(Y = c \mid \mathbf{x}) = 1$

- **Función de costo (Cross-Entropy):**

$$J(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} y_{ic} \log f_c(\mathbf{x}_i)$$

  - $y_{ic} = 1$ si $y_i = c$, 0 en otro caso (one-hot)

k-Nearest Neighbors (kNN) (1/3)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- **Idea:**
  - Para clasificar una nueva observación $\mathbf{x}$, buscamos los k ejemplos de entrenamiento más cercanos (distancia geométrica)
  - La predicción se basa en las clases de esos vecinos
- **En la formalización general:**
  - Entrada: $\mathbf{x} \in \mathbb{R}^d$
  - Salida: $f(\mathbf{x}) = (f_1(\mathbf{x}), \ldots, f_C(\mathbf{x}))$
  - En kNN, $f_c(\mathbf{x})$ puede interpretarse como:

$$f_c(\mathbf{x}) = \frac{\#\{\text{vecinos de clase } c\}}{\mathrm{k}}$$

- **Regla de decisión:**

$$\hat{y} = \arg\max_c f_c(\mathbf{x})$$

  - La clase predicha es la más frecuente entre los vecinos más cercanos

k-Nearest Neighbors (kNN) (2/3)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

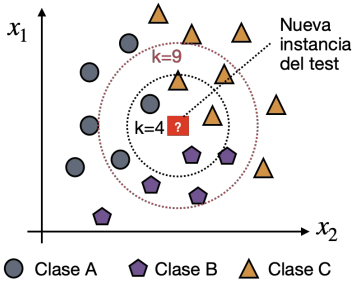
- **Aprendizaje del modelo:**
  - kNN no ajusta pesos ni coeficientes (no paramétrico)
  - kNN "aprende" $\rightarrow$ memoriza el dataset de entrenamiento + regla de #(vecinos).
- **Hiperparámetro principal:**
  - k: número de vecinos considerados

k **pequeño**	k **grande**
Modelo más flexible	Modelo más estable
Sensible al ruido	Menos sensible al ruido
Riesgo de sobreajuste	Riesgo de subajuste
Frontera de decisión irregular	Frontera de decisión más suave

- **Importante en la práctica:**
  - La elección de la distancia importa
  - La escala de las variables importa $\Rightarrow$ conviene normalizar

k-Nearest Neighbors (kNN) (3/3)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas



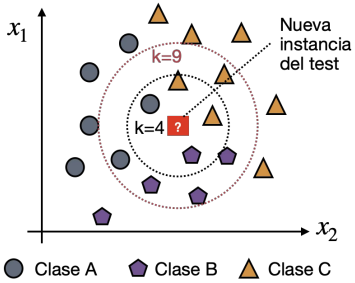
Evaluamos a qué clase [A,B,C] pertenece ■

- Para $\mathrm{k} = 4$
  - $f(■) = (f_{\mathrm{A}}(■), f_{\mathrm{B}}(■), f_{\mathrm{C}}(■)) = \left(\frac{1}{4}, \frac{1}{4}, \frac{2}{4}\right)$
  - $\hat{y} = \arg\max_c f_c(\mathbf{x}) \equiv \mathrm{C}$
  - ■ pertenece a C
- Para $\mathrm{k} = 9$
  - $f(■) = (f_{\mathrm{A}}(■), f_{\mathrm{B}}(■), f_{\mathrm{C}}(■)) = \left(\frac{2}{9}, \frac{4}{9}, \frac{3}{9}\right)$
  - $\hat{y} = \arg\max_c f_c(\mathbf{x}) \equiv \mathrm{B}$
  - ■ pertenece a B

k-Nearest Neighbors (kNN) (3/3)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

FCEyN



Evaluamos a qué clase [A,B,C] pertenece ■

- Para $\mathrm{k} = 4$
  - $f(\blacksquare) = (f_{\mathrm{A}}(\blacksquare), f_{\mathrm{B}}(\blacksquare), f_{\mathrm{C}}(\blacksquare)) = \left(\frac{1}{4}, \frac{1}{4}, \frac{2}{4}\right)$
  - $\hat{y} = \arg\max_c f_c(\mathbf{x}) \equiv \mathrm{C}$
  - ■ pertenece a C
- Para $\mathrm{k} = 9$
  - $f(\blacksquare) = (f_{\mathrm{A}}(\blacksquare), f_{\mathrm{B}}(\blacksquare), f_{\mathrm{C}}(\blacksquare)) = \left(\frac{2}{9}, \frac{4}{9}, \frac{3}{9}\right)$
  - $\hat{y} = \arg\max_c f_c(\mathbf{x}) \equiv \mathrm{B}$
  - ■ pertenece a B

Para considerar

- En la práctica, tendríamos que calcular todas las distancias de cada punto de entrenamiento a ■ $\rightarrow$ si el dataset es muy grande, kNN es ineficiente
- De la elección de k depende la clasificación y no existe regla general para elegir el mejor k dado un dataset.

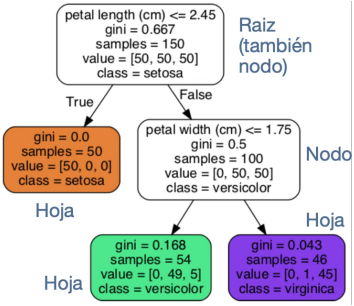
Árboles de decisión (1/3)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- **Idea:**
  - Clasificar mediante una secuencia de reglas binarias
  - Cada regla divide el espacio de datos
- **Estructura:**
  - Nodos internos: reglas (divisiones)
  - Hojas: predicción final
- **En la formalización general:**
  - Entrada: $\mathbf{x} \in \mathbb{R}^d$
  - Salida: $f(\mathbf{x}) = (f_1(\mathbf{x}), \ldots, f_C(\mathbf{x}))$
  - En árboles $f_c(\mathbf{x})$: proporción de ejemplos de clase c en la hoja
- **Regla de decisión:**

$$\hat{y} = \arg\max_c f_c(\mathbf{x})$$

Objetivo: separar mejor las clases en cada paso



Ejemplo de árbol de decisión para el dataset Iris

- El modelo elige automáticamente:
  - qué variable usar (x_j)
  - en qué valor dividir

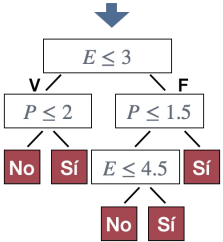
Árboles de decisión (2/3)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

FCEyN

Ejemplo: construcción de un árbol a partir del conjunto de entrenamiento

Features			Clase
ID	E	P	Reactivo
A	2	1	No
B	3	3	Sí
C	4	1	No
D	6	2	Sí
E	7	3	Sí
F	5	1	Sí



1. Comenzar con todos los datos en un nodo (raíz)
2. Repetir:
   - Probar diferentes divisiones:
     - elegir una variable (x_j) $\rightarrow$ e.g. E
     - elegir un umbral (t) $\rightarrow$ e.g. $E > 2$
   - Evaluar qué tan bien separa las clases (ej: Gini)
   - Elegir la mejor división
   - Dividir el dataset en dos nodos
3. Condición de parada:
   - Si el nodo es suficientemente puro (quedan clases bien separadas) $\rightarrow$ hoja
   - Si no $\rightarrow$ repetir el proceso en cada nodo hijo

Árboles de decisión (3/3)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

FCEyN

- **Aprendizaje del modelo:**
  - Aprende una estructura de decisiones
  - Parámetros:
    - variable en cada nodo
    - umbral de división (si numérico)
    - estructura del árbol
    - predicción en cada hoja
- **Hiperparámetros importantes:**
  - Profundidad máxima del árbol

Árbol profundo	**Árbol poco profundo**
Muy flexible	Más simple
Puede memorizar los datos	Puede no capturar patrones
Riesgo de sobreajuste	Riesgo de subajuste
Fronteras de decisión complejas	Fronteras de decisión simples

  - Cantidad mínima de muestras por nodo
- **Importante en la práctica:**
  - No requiere normalización
  - Fácil de interpretar

Bosques aleatorios (Random Forest) (1/3)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- **Idea:**
  - Combinar muchos T árboles de decisión
  - Cada árbol t hace una predicción
  - El modelo final combina esas predicciones
- **En la formalización general:**
  - Entrada: $\mathbf{x} \in \mathbb{R}^d$
  - Cada árbol lleva a una hoja y produce:

$$f^{(t)}(\mathbf{x}) = (f_1^{(t)}(\mathbf{x}), \ldots, f_C^{(t)}(\mathbf{x}))$$

    - donde $f_c^{(t)}(\mathbf{x})$ es la proporción de clase c en esa hoja
  - El modelo final promedia:

$$f(\mathbf{x}) = \frac{1}{T} \sum_{t=1}^{T} f^{(t)}(\mathbf{x})$$

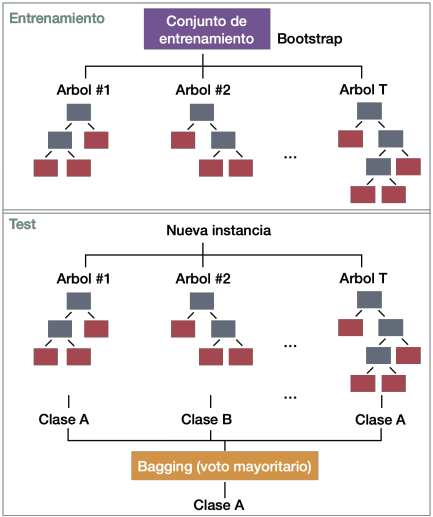
- **Regla de decisión:**

$$\hat{y} = \arg\max_c f_c(\mathbf{x})$$

Bosques aleatorios (Random Forest) (2/3)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- **Problema de un solo árbol:**
  - Alta varianza (sensible a los datos)
  - Alto riesgo de sobreajuste
- **Idea clave:**
  - Crear árboles distintos e independientes
  - Promediar sus predicciones
- **Cómo se generan árboles distintos:**
  - Cada árbol usa un subconjunto de los datos (bootstrap)
  - Cada división usa un subconjunto de variables



Bosques aleatorios (Random Forest) (3/3)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- **Hiperparámetros importantes:**
  - Número de árboles (T)
    - Más árboles: más estable, menor varianza
  - Profundidad máxima de cada árbol
    - Árboles profundos: más flexibles, posible sobreajuste
  - Hay más, pero no los vamos a estudiar en este curso por simplicidad
- **Importante:**
  - No requiere normalización
  - Suele funcionar bien sin mucho ajuste
  - Reduce la varianza, mejora la generalización

Impulso por gradiente (Gradient Boosting) (1/3)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

FCEyN

- **Idea:**
  - Construir el modelo **de forma aditiva** hasta T
  - Cada nuevo modelo $t \in T$ intenta corregir los errores del anterior
- **En la formalización general:**
  - Modelo como suma de árboles correctivos:

$$f(\mathbf{x}) = \sum_{t=1}^{T} \eta \, f^{(t)}(\mathbf{x})$$

    - $f^{(t)}$: árbol en la iteración t
    - η: learning rate (qué tanto aporta cada árbol)
  - Cada nuevo árbol se entrena para aproximar los errores del modelo actual
- **Regla de decisión:**

$$\hat{y} = \arg\max_{c} f_c(\mathbf{x})$$

  - $f_c(\mathbf{x})$: suma de aportes de todos los árboles sobre la clase c

Impulso por gradiente (Gradient Boosting) (2/3)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- **Proceso iterativo:**
  - Inicializar modelo simple
  - Repetir:
    - Evaluar errores del modelo actual
    - Entrenar un nuevo árbol para corregir esos errores
    - Agregarlo al modelo
- **Intuición:**
  - Cada modelo se enfoca en lo que el anterior hizo mal ("errores residuales")

GBoost se usa en regresión también...

...y es más sencillo de entender con números y no categorías. Vamos a ver este algoritmo en la parte II de aprendizaje supervisado en más detalle.

Impulso por gradiente (Gradient Boosting) (3/3)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- **Hiperparámetros importantes:**
  - Número de árboles (T)
  - Learning rate (η)
    - Learning rate pequeño: aprendizaje más lento pero mejor generalización
  - Profundidad de cada árbol
- **Importante:**
  - Muy potente, pero sensible a hiperparámetros
  - Tendencia a sobreajustar
  - Reduce sesgo

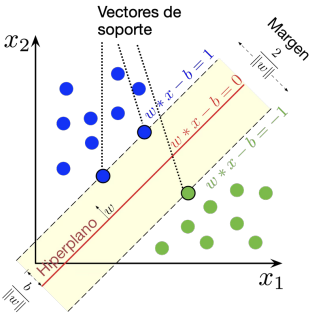
Diferencia entre RF y GB

- Bosques aleatorios (RF) promedia predicciones de T modelos independientes
- Gradient Boosting (GBoost) construye T modelos dependientes que corrigen errores

Support Vector Machines (SVM) (1/3)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

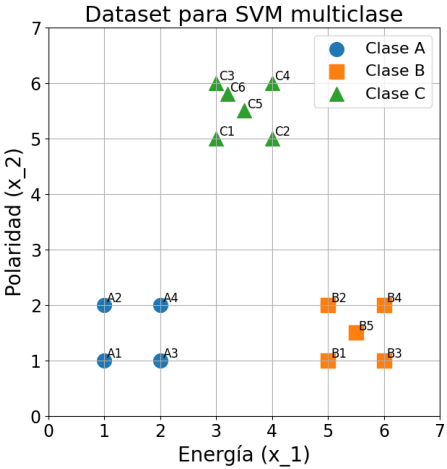
- **Idea para el caso lineal:**
  - Para cada clase c, aprendemos un hiperplano:
    $$f_c(\mathbf{x}) = \mathbf{w}_c^\top \mathbf{x} + b_c$$
    que maximiza el margen entre las clases.
  - Margen: distancia entre la frontera de decisión y los puntos más cercanos.
- **En la formalización general:**
  - Entrada: $\mathbf{x} \in \mathbb{R}^d$
  - Salida:
    $$f(\mathbf{x}) = (f_1(\mathbf{x}), \ldots, f_C(\mathbf{x}))$$
  - $f_c(\mathbf{x})$ mide qué tan compatible es $\mathbf{x}$ con la clase c
- **Regla de decisión:**
  $$\hat{y} = \arg\max_c f_c(\mathbf{x})$$



Ejemplo de SVM para el caso binario

Support Vector Machines (SVM) (2/3)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas



- Estrategia: One-vs-Rest, entrenamos un clasificador por clase:
  - Clase A vs (B + C)
  - Clase B vs (A + C)
  - Clase C vs (A + B)
- Nueva molécula: $\mathbf{x}_N = (3.2,\ 4.8)$
- Hiperplanos (aprendidos):

$$
\begin{aligned}
f_A(\mathbf{x}) &= -x_1 - 0.5x_2 + 4.2 &&= -1.4 \\
f_B(\mathbf{x}) &= x_1 - 0.8x_2 + 0.04 &&= -0.6 \\
f_C(\mathbf{x}) &= 0.3x_1 + x_2 - 3.66 &&= 2.1
\end{aligned}
$$

$$f(\mathbf{x}) = (-1.4,\ -0.6,\ 2.1) \quad \Rightarrow \quad \hat{y} = C$$

Support Vector Machines (SVM) (3/3)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

FCEyN

- **Aprendizaje del modelo**
  - Parámetros: $\mathbf{w}, b$ que definen el hiperplano.
- **Hiperparámetros importantes**
  - C: controla el trade-off entre margen y errores
    - C grande: menos errores, menor margen (riesgo de sobreajuste)
    - C chico: permite errores, mayor margen (mejor generalización)
- **Importante**
  - El modelo depende solo de los vectores de soporte
  - Sensible a la escala $\Rightarrow$ normalizar datos

¿Y si los datos no son linealmente separables?

- Usamos **kernels**
  - Permiten aprender fronteras no lineales
  - Idea: transformar los datos a otro espacio donde sí sean separables

Métricas de evaluación en clasificación (1/2)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- **Matriz de confusión (para una clase c):**
  - TP (True Positive): predije c y era c
  - FP (False Positive): predije c pero no era c
  - FN (False Negative): no predije c pero era c
  - TN (True Negative): no predije c y no era c
- **Accuracy (Exactitud):**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

  - Proporción de predicciones correctas
  - Mide desempeño global
- **Precision (para clase c):**

$$\text{Precision}_c = \frac{TP}{TP + FP}$$

  - De lo que predije como clase c, ¿cuánto era correcto?

Métricas de evaluación en clasificación (2/2)

Introducción
Herramientas de entrenamiento
Optimización
BGD
SGD
MBGD
Conjunto de validación
Parámetros vs Hiperparámetros
Modelos de Clasificación
Regresión logística
kNN
Árboles de decisión
Impulso por gradiente
SVM
Métricas

- **Recall (para clase c):**

$$\text{Recall}_c = \frac{TP}{TP + FN}$$

  - De los que eran clase c, ¿cuántos detecté?

- **F1-score (para clase c):**

$$F1_c = 2 \cdot \frac{\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}$$

  - Balance entre precision y recall

Multiclase

Promediamos sobre clases. Ejemplo:

- **Macro F1:** promedio simple de $F1_c$
- **Weighted F1:** ponderado por tamaño de clase

Gracias

FACULTAD DE CIENCIAS
EXACTAS Y NATURALES
Ciencia, Tecnología y Humanismo